Linear Regression and Logistic Regression

Student Name: Evelyn (CUI CONGWEN)

Student ID: P2321167

Exercise 1: Linear regression

In this homework, we investigated multivariate linear regression using gradient descent and stochastic gradient descent. The dataset consisted of apartment prices in Haizhu District, Guangzhou, with **two input features:** apartment size (square meters) and distance to the district center (kilometers). The **target variable** was the apartment price (billion RMB).
According to the equation of Multiple Linear Regression:

$$y = w_0 + w_1x_1 + w_2 x_2 + ... + w_n*x_n$$

The model parameters included:
w_0: intercept term
w_1: coefficient for apartment size
w_2: coefficient for distance to center
Thus, the regression model tuned **three parameters** in total.

Writing Sample CUI Congwen

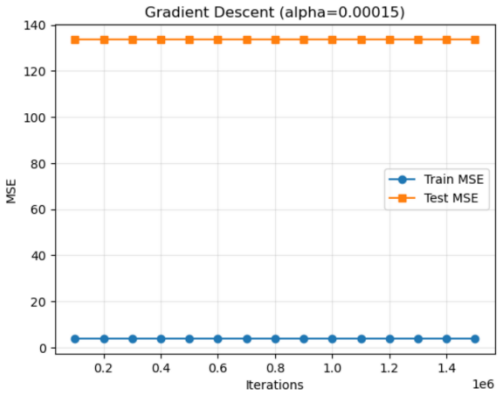
Experiment 1.1: Gradient Descent

According to the requirement, we need to initialize learning rate α = 0.00015, iterations = 1,500,000, parameters = 0.
The corresponding code is shown below:

```
w = np.zeros(x_train.shape[1])
alpha = 0.00015
iterations = 1500000
```

A total of 1,500,000 iterations were performed, and the training and testing errors were recorded at every 100,000 iterations. The results are summarized as follows.

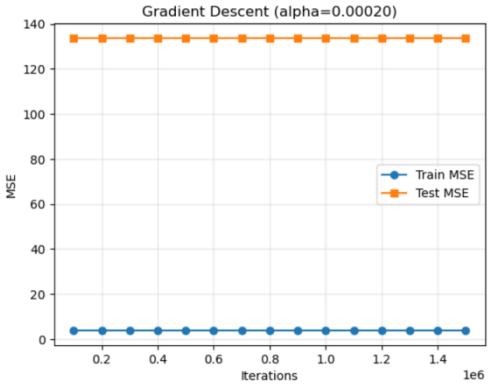
```
Iter 100000: Train MSE=3.82, Test MSE=133.74
Iter 200000: Train MSE=3.82, Test MSE=133.75
Iter 300000: Train MSE=3.82, Test MSE=133.75
Iter 400000: Train MSE=3.82, Test MSE=133.75
Iter 500000: Train MSE=3.82, Test MSE=133.75
Iter 600000: Train MSE=3.82, Test MSE=133.75
Iter 700000: Train MSE=3.82, Test MSE=133.75
Iter 800000: Train MSE=3.82, Test MSE=133.75
Iter 900000: Train MSE=3.82, Test MSE=133.75
Iter 1000000: Train MSE=3.82, Test MSE=133.75
Iter 1100000: Train MSE=3.82, Test MSE=133.75
Iter 1200000: Train MSE=3.82, Test MSE=133.75
Iter 1300000: Train MSE=3.82, Test MSE=133.75
Iter 1400000: Train MSE=3.82, Test MSE=133.75
Iter 1500000: Train MSE=3.82, Test MSE=133.75
```



According to the figure, we found that both training and testing MSE stabilized before 200,000 iterations, indicating convergence was achieved early.

Following the hint of the homework to change Alpha = 0.00020

```
Iter 100000: Train MSE=3.82, Test MSE=133.75
Iter 200000: Train MSE=3.82, Test MSE=133.75
Iter 300000: Train MSE=3.82, Test MSE=133.75
Iter 400000: Train MSE=3.82, Test MSE=133.75
Iter 500000: Train MSE=3.82, Test MSE=133.75
Iter 600000: Train MSE=3.82, Test MSE=133.75
Iter 700000: Train MSE=3.82, Test MSE=133.75
Iter 800000: Train MSE=3.82, Test MSE=133.75
Iter 900000: Train MSE=3.82, Test MSE=133.75
Iter 1000000: Train MSE=3.82, Test MSE=133.75
Iter 1100000: Train MSE=3.82, Test MSE=133.75
Iter 1200000: Train MSE=3.82, Test MSE=133.75
Iter 1300000: Train MSE=3.82, Test MSE=133.75
Iter 1400000: Train MSE=3.82, Test MSE=133.75
Iter 1500000: Train MSE=3.82, Test MSE=133.75
```



We observed that the MSE did not change significantly, as the learning rate (α) determines the step size toward the optimal point. The results suggest that the model had already converged before iteration 100,000.

To make this relationship clearer, we can further demonstrate it by setting a smaller learning rate.

Alpha = 0.00003

```
Iter 100000: Train MSE=126.22, Test MSE=56.75
Iter 200000: Train MSE=4.25, Test MSE=125.32
Iter 300000: Train MSE=3.82, Test MSE=133.23
Iter 400000: Train MSE=3.82, Test MSE=133.72
Iter 500000: Train MSE=3.82, Test MSE=133.74
Iter 600000: Train MSE=3.82, Test MSE=133.75
Iter 700000: Train MSE=3.82, Test MSE=133.75
Iter 800000: Train MSE=3.82, Test MSE=133.75
Iter 900000: Train MSE=3.82, Test MSE=133.75
Iter 1000000: Train MSE=3.82, Test MSE=133.75
Iter 1100000: Train MSE=3.82, Test MSE=133.75
Iter 1200000: Train MSE=3.82, Test MSE=133.75
Iter 1300000: Train MSE=3.82, Test MSE=133.75
Iter 1400000: Train MSE=3.82, Test MSE=133.75
Iter 1500000: Train MSE=3.82, Test MSE=133.75
```

Alpha = 0.00006

```
Iter 100000: Train MSE=4.25, Test MSE=125.32
Iter 200000: Train MSE=3.82, Test MSE=133.72
Iter 300000: Train MSE=3.82, Test MSE=133.75
Iter 400000: Train MSE=3.82, Test MSE=133.75
Iter 500000: Train MSE=3.82, Test MSE=133.75
Iter 600000: Train MSE=3.82, Test MSE=133.75
Iter 700000: Train MSE=3.82, Test MSE=133.75
Iter 800000: Train MSE=3.82, Test MSE=133.75
Iter 900000: Train MSE=3.82, Test MSE=133.75
Iter 1000000: Train MSE=3.82, Test MSE=133.75
Iter 1100000: Train MSE=3.82, Test MSE=133.75
Iter 1200000: Train MSE=3.82, Test MSE=133.75
Iter 1300000: Train MSE=3.82, Test MSE=133.75
Iter 1400000: Train MSE=3.82, Test MSE=133.75
Iter 1500000: Train MSE=3.82, Test MSE=133.75
```

Alpha = 0.00009

```
Iter 100000: Train MSE=3.82, Test MSE=133.23
Iter 200000: Train MSE=3.82, Test MSE=133.75
Iter 300000: Train MSE=3.82, Test MSE=133.75
Iter 400000: Train MSE=3.82, Test MSE=133.75
Iter 500000: Train MSE=3.82, Test MSE=133.75
Iter 600000: Train MSE=3.82, Test MSE=133.75
Iter 700000: Train MSE=3.82, Test MSE=133.75
Iter 800000: Train MSE=3.82, Test MSE=133.75
Iter 900000: Train MSE=3.82, Test MSE=133.75
Iter 1000000: Train MSE=3.82, Test MSE=133.75
Iter 1100000: Train MSE=3.82, Test MSE=133.75
Iter 1200000: Train MSE=3.82, Test MSE=133.75
Iter 1300000: Train MSE=3.82, Test MSE=133.75
Iter 1400000: Train MSE=3.82, Test MSE=133.75
Iter 1500000: Train MSE=3.82, Test MSE=133.75
```

Alpha = 0.00012

```
Iter 100000: Train MSE=3.82, Test MSE=133.72
Iter 200000: Train MSE=3.82, Test MSE=133.75
Iter 300000: Train MSE=3.82, Test MSE=133.75
Iter 400000: Train MSE=3.82, Test MSE=133.75
Iter 500000: Train MSE=3.82, Test MSE=133.75
Iter 600000: Train MSE=3.82, Test MSE=133.75
Iter 700000: Train MSE=3.82, Test MSE=133.75
Iter 800000: Train MSE=3.82, Test MSE=133.75
Iter 900000: Train MSE=3.82, Test MSE=133.75
Iter 1000000: Train MSE=3.82, Test MSE=133.75
Iter 1100000: Train MSE=3.82, Test MSE=133.75
Iter 1200000: Train MSE=3.82, Test MSE=133.75
Iter 1300000: Train MSE=3.82, Test MSE=133.75
Iter 1400000: Train MSE=3.82, Test MSE=133.75
Iter 1500000: Train MSE=3.82, Test MSE=133.75
```

Writing Sample © Li Congwen

According to the figure, we found that as the **learning rate increases**, the model **converges to the optimal point more quickly**. So, **fewer iterations** are required

In other words, when the **learning rate is small, more iterations are required** for the model to stabilize. For example, when α = 0.00003, we observed that it took significantly more iterations to converge to the optimal point.

This finding highlights that with a small learning rate, the number of iterations must be increased; Otherwise, the model may fail to reach the optimal solution.

Based on the previous data, we observed that as the learning rate increases, the number of iterations required for convergence is reduced, showing a negative correlation between these two factors.

Since the number of iterations represents the amount of training that directly influences computational efficiency, our goal is to minimize it as much as possible.

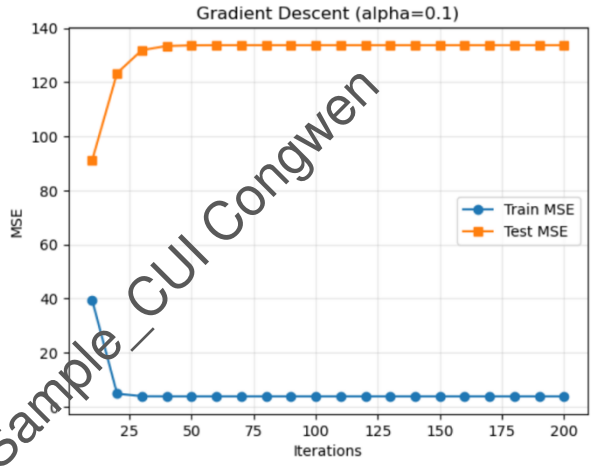
This raises an important question: *given that a larger learning rate reduces the required iterations, can we set the learning rate to a very high value in order to further decrease the number of iterations?*

We now proceed to validate this hypothesis:

We increased the learning rate as much as possible and employed more frequent recording intervals in order to precisely identify the range of iterations required for convergence

Setting the learning rate = 0.1, record per 100, it only takes less than 150 iteration to get stable

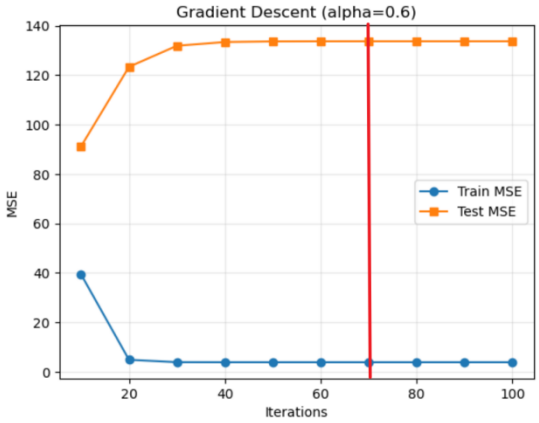
```
Iter 10: Train MSE=6608.10, Test MSE=5634.99
Iter 20: Train MSE=684.33, Test MSE=245.44
Iter 30: Train MSE=95.56, Test MSE=61.50
Iter 40: Train MSE=16.52, Test MSE=94.16
Iter 50: Train MSE=5.58, Test MSE=117.28
Iter 60: Train MSE=4.06, Test MSE=127.37
Iter 70: Train MSE=3.85, Test MSE=131.34
Iter 80: Train MSE=3.82, Test MSE=132.84
Iter 90: Train MSE=3.82, Test MSE=133.41
Iter 100: Train MSE=3.82, Test MSE=133.62
Iter 110: Train MSE=3.82, Test MSE=133.70
Iter 120: Train MSE=3.82, Test MSE=133.73
Iter 130: Train MSE=3.82, Test MSE=133.74
Iter 140: Train MSE=3.82, Test MSE=133.74
Iter 150: Train MSE=3.82, Test MSE=133.75
Iter 160: Train MSE=3.82, Test MSE=133.75
Iter 170: Train MSE=3.82, Test MSE=133.75
Iter 180: Train MSE=3.82, Test MSE=133.75
Iter 190: Train MSE=3.82, Test MSE=133.75
Iter 200: Train MSE=3.82, Test MSE=133.75
```



Writing Sample _ CUI Congwen

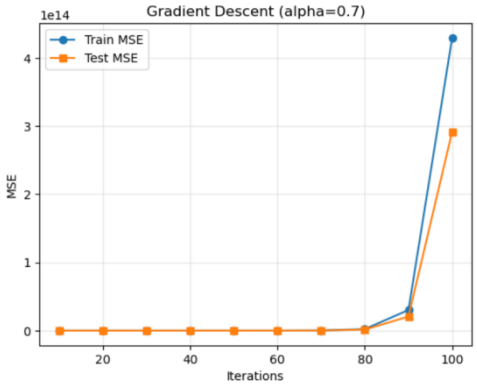
Setting the learning rate 0.6, record per 10, it only takes less than 70 iteration to get stable

```
Iter 10: Train MSE=39.38, Test MSE=91.21
Iter 20: Train MSE=4.80, Test MSE=123.40
Iter 30: Train MSE=3.84, Test MSE=131.93
Iter 40: Train MSE=3.82, Test MSE=133.44
Iter 50: Train MSE=3.82, Test MSE=133.70
Iter 60: Train MSE=3.82, Test MSE=133.74
Iter 70: Train MSE=3.82, Test MSE=133.75
Iter 80: Train MSE=3.82, Test MSE=133.75
Iter 90: Train MSE=3.82, Test MSE=133.75
Iter 100: Train MSE=3.82, Test MSE=133.75
```



Changing the learning rate into 0.7, with the same iteration of 100, if the hypothesis is true, there should be less than 70 iteration to make it stable.

```
Iter 10: Train MSE=18212.56, Test MSE=11016.31
Iter 20: Train MSE=258709.79, Test MSE=170305.76
Iter 30: Train MSE=3675642.86, Test MSE=2476812.39
Iter 40: Train MSE=52222694.91, Test MSE=35410307.93
Iter 50: Train MSE=741968796.10, Test MSE=503937290.17
Iter 60: Train MSE=10541733438.56, Test MSE=7162986062.03
Iter 70: Train MSE=149774687247.92, Test MSE=101782059493.88
Iter 80: Train MSE=2127966626928.72, Test MSE=1446142489724.83
Iter 90: Train MSE=30233693347170.55, Test MSE=20546651225281.86
Iter 100: Train MSE=429553829390380.56, Test MSE=291923049825039.88
```



But according to the data we obtained, the results became more unstable and fluctuated rather than converging smoothly.

Keeping the learning rate at 0.7 while increasing the number of iterations:

```
Iter 1000: Train MSE=22929406034718370326564164211584323992934349437029203469064027752807987865504493275419937280203677424625811981042974720.00, Test MS
E=15582744129766339634010101687069887140852968828525698539236123598447372431002064521464930109864892248816974417499258880.00
Iter 2000: Train MSE=41023426001608403193855224999584418961957698265304816568187106996345101539020238147800738638398986110056707047230113288697088274127
099562070544150117566288674681366732358973313967511797197725965722068717701400604710592592627146057318 4.00, Test MSE=27879376802937701481372362507056016
504174415685855318825522033718414180858376740037232207327209526677003960712302670856816577562320959471280164652661059961634025196034841089781895614994 8
992841954930356322201425368360866468959910625 8.00
Iter 3000: Train MSE=inf, Test MSE=inf
Iter 4000: Train MSE=inf, Test MSE=inf
Iter 5000: Train MSE=inf, Test MSE=inf
Iter 6000: Train MSE=nan, Test MSE=nan
Iter 7000: Train MSE=nan, Test MSE=nan
Iter 8000: Train MSE=nan, Test MSE=nan
Iter 9000: Train MSE=nan, Test MSE=nan
Iter 10000: Train MSE=nan, Test MSE=nan
```

```
C:\Users\Asus\AppData\Local\Temp\ipykernel_12664\1088403691.py:13: RuntimeWarning: overflow encoun
tered in square
  train_mse = np.mean((x_train.dot(w) - y_train) ** 2)
C:\Users\Asus\AppData\Local\Temp\ipykernel_12664\1088403691.py:14: RuntimeWarning: overflow encoun
tered in square
  test_mse = np.mean((x_test.dot(w) - y_test) ** 2)
```

Writing Sample_CUI Congwen

We found the MSE diverging to infinity or producing NaN values, indicating a clear divergence of the training process. This demonstrates that an excessively large step size may cause the parameters to oscillate around the optimal point or even diverge.

Conclusion

Based on the above experiments, we draw the following conclusions:

Learning rate cannot be increased indefinitely; Otherwise, convergence will be compromised.

Within a reasonable range, increasing the **learning rate does reduce the number of iterations required** and thereby improves training efficiency.

Beyond a critical threshold, however, an excessively large learning rate **destabilizes the model, leading to oscillations or even divergence.**

Therefore, the relationship between learning rate and iteration count is nonlinear: there exists an optimal interval that achieves both rapid convergence and stable performance.

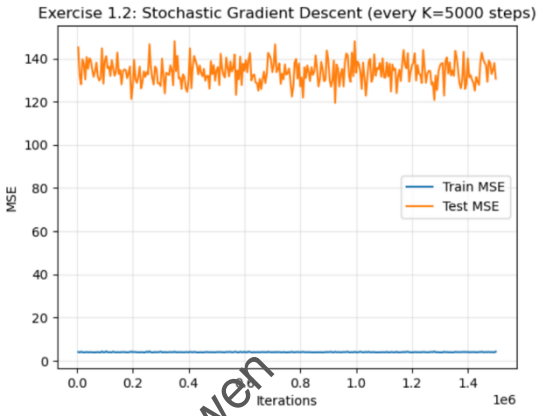
Experiment 1.2: Stochastic Gradient Descent

Implement SGD by initialize learning rate = 0.01, Recording interval(K) = 5000, and maximum steps = 1500000.

```
alpha_sgd = 0.01
K = 5000
max_steps = 1500000
```

The obtained results are illustrated in the following figure:

```
Step 100000: Train MSE=3.88, Test MSE=138.86
Step 200000: Train MSE=3.88, Test MSE=127.00
Step 300000: Train MSE=4.16, Test MSE=127.25
Step 400000: Train MSE=3.84, Test MSE=130.41
Step 500000: Train MSE=3.96, Test MSE=134.60
Step 600000: Train MSE=4.05, Test MSE=134.30
Step 700000: Train MSE=3.93, Test MSE=133.68
Step 800000: Train MSE=3.87, Test MSE=127.10
Step 900000: Train MSE=3.93, Test MSE=139.42
Step 1000000: Train MSE=3.87, Test MSE=133.42
Step 1100000: Train MSE=3.91, Test MSE=131.31
Step 1200000: Train MSE=3.84, Test MSE=136.73
Step 1300000: Train MSE=3.83, Test MSE=136.58
Step 1400000: Train MSE=4.02, Test MSE=140.01
Step 1500000: Train MSE=4.14, Test MSE=130.63
```



Writing Sample CUI Congwen

According to the figure, we found that the **training** mean squared error (MSE) converges to a **stable value more rapidly**, whereas the **testing** error **continues to fluctuate** throughout the iterations.

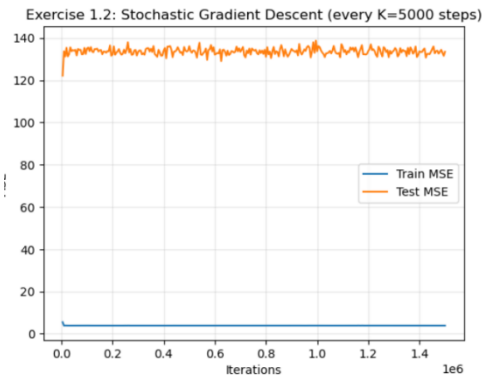
This phenomenon has explained the main difference between SGD and GD.

SGD updates the parameters using only one sample at each iteration. The training error is computed as the average over the entire training set, so its overall trend gradually decreases and stabilizes.

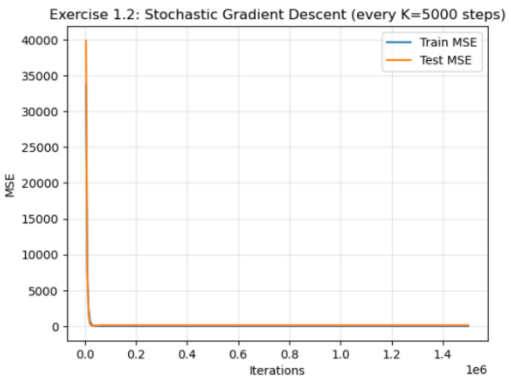
In contrast, the testing error is more sensitive, as the model parameters are continuously influenced by individual samples, leading to persistent oscillations in the performance on the test set.

To examine the relationship between changes in the learning rate and the fluctuations observed in the curves, we change the value of learning rate.

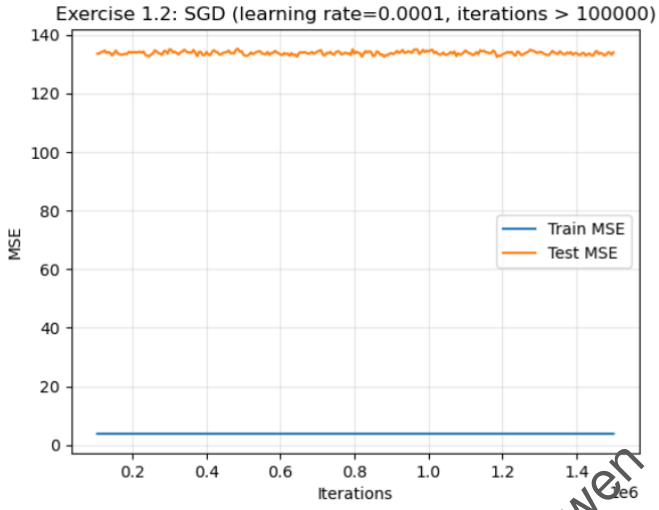
Learning rate = 0.001



Learning rate = 0.0001



With a **smaller learning rate**, the **initial fluctuations in SGD become more pronounced.** However, in the later stages, the magnitude of the errors differs substantially across settings (e.g., values around 100 versus 10,000), making it difficult to directly compare the oscillations from the figures. To address this, we focus on the segment of the curve with learning rate = 0.001, specifically for iterations greater than 0.1 million, in order to provide a clearer comparison



Based on the figure, we observe that the fluctuations become smaller in the later stages. Consequently, we conclude that as the learning rate decreases, SGD exhibits large oscillations at the beginning but gradually stabilizes over time.

Because SGD updates the parameters using only one sample at a time, which inherently introduces randomness into the gradient direction. When the learning rate is large, this randomness is amplified, resulting in persistent oscillations.

In contrast, with a smaller learning rate, although convergence is slower in the early stages, the smaller update steps eventually smooth out the stochastic fluctuations, leading to more stable behavior in the later iterations.

Writing Sample CUI Congwen

Conclusion

Regardless of whether using SGD or batch gradient descent, the conclusion remains the same: with a smaller learning rate, the step size becomes shorter, requiring more iterations to reach convergence. In other words, convergence is slower.

However, **compared to batch gradient descent**, stochastic gradient descent exhibits more pronounced fluctuations in the early stage due to its single-sample updates. While GD produces a smooth and steadily decreasing error curve, SGD shows oscillations around the optimal solution before gradually stabilizing. This difference highlights the inherent randomness of SGD, which contrasts with the deterministic trajectory of GD.

Exercise 2: Logistic Regression

(a) Conditional Log-Likelihood

Suppose we have n training examples, each with p features.

- Feature vector: $x^{(i)}=(x_1^{(i)},x_2^{(i)},\ldots,x_p^{(i)})$
- Label: $y^{(i)}\in\{0,1\}$
- Parameters: $w=(w_0,w_1,\ldots,w_p)$

The logistic regression model predicts:

$$h_w(x^{(i)}) = \frac{1}{1+e^{-w^T x^{(i)}}}$$

The conditional log-likelihood of the training data is:

$$\ell(w) = \sum_{i=1}^{n}\left[y^{(i)} \cdot \log(h_w(x^{(i)})) + (1-y^{(i)}) \cdot \log(1-h_w(x^{(i)}))\right]$$

(b) Gradient Derivation

For each parameter w_j, the partial derivative of the log-likelihood is:

$$\frac{\partial \ell(w)}{\partial w_j} = \sum_{i=1}^{n}\left(y^{(i)} - h_w(x^{(i)})\right) \cdot x_j^{(i)}$$

For the bias term w_0:

$$\frac{\partial \ell(w)}{\partial w_0} = \sum_{i=1}^{n}\left(y^{(i)} - h_w(x^{(i)})\right)$$

Writing Sample — CUI Congwen

(c) Training the Classifier:

The logistic regression classifier was trained using the dataForTrainingLogistic.txt. The first six columns were treated as **features**, and the last column was the **binary label.**

All parameters were initialized to zero.Training was conducted using stochastic gradient ascent with a initialized learning rate of 0.001 for 20,000 iterations.

```
w = np.zeros(x_train.shape[1])
alpha = 0.001
iterations = 20000
```

Training was conducted using stochastic gradient ascent (SGA). At each iteration, the prediction probability was computed as:

$$h_w(x^{(i)}) = \frac{1}{1+e^{-w^T x^{(i)}}}$$

The parameters were updated according to the rule:

$$w \leftarrow w + \alpha \cdot (y^{(i)} - h_w(x^{(i)})) \cdot x^{(i)}$$

where α is the learning rate.

After training, the algorithm converged to an optimal parameter vector *w*. Using these parameters, predicted labels for the testing dataset dataForTestingLogistic.txt were calculated with a threshold of 0.5. The number of misclassified examples and the test accuracy were reported.

To evaluate the influence of the learning rate, I experimented with several values of α.

(1) Learning rate = 0.001, Iteration = 20000

```
Number of misclassified: 0 / 100
w: [ 1.35235471 -7.52652413 11.17525323 -7.87297515  9.68860677 -5.36582552
  0.07294699]
Test accuracy: 1.0
```

(2) Learning rate = 0.01, Iteration = 10000

```
Number of misclassified: 0 / 100
w: [  1.56473025 -10.51303576  16.39091991 -11.54346862  13.00521872
  -6.43475541  -0.04583008]
Test accuracy: 1.0
```

(3) Learning rate = 0.01, Iteration =1000

```
Number of misclassified: 0 / 100
w: [ 1.24035178 -6.50156671  9.4115221  -6.70151707  8.43257101 -4.83320413
  0.12537571]
Test accuracy: 1.0
```

(4) Learning rate = 0.1, Iteration = 100

```
Number of misclassified: 0 / 100
w: [ 1.22739592 -6.5174857   9.4390119  -6.73563277  8.42344253 -4.82273433
  0.12007552]
Test accuracy: 1.0
```

(5) Learning rate = 1, Iteration = 10

```
Number of misclassified: 0 / 100
w: [ 1.31970558 -6.58230481  9.86637325 -6.80655936  8.60531374 -4.49103856
  0.2319089 ]
Test accuracy: 1.0
```

Writing Sample _ CUI Congwen

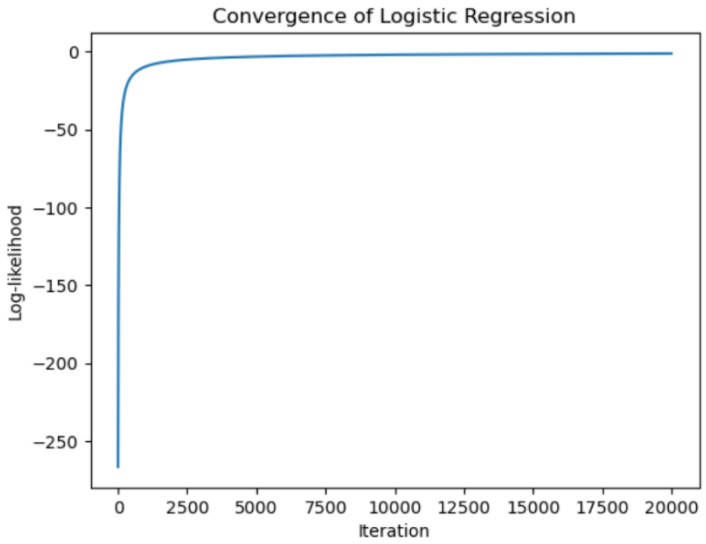
The experimental results are summarized in the following table:

Learning rate (α)	Iterations	Accuracy	Misclassified
0.001	20000	1.0	0
0.01	10000	1.0	0
0.01	1000	1.0	0
0.1	100	1.0	0
1	10	1.0	0

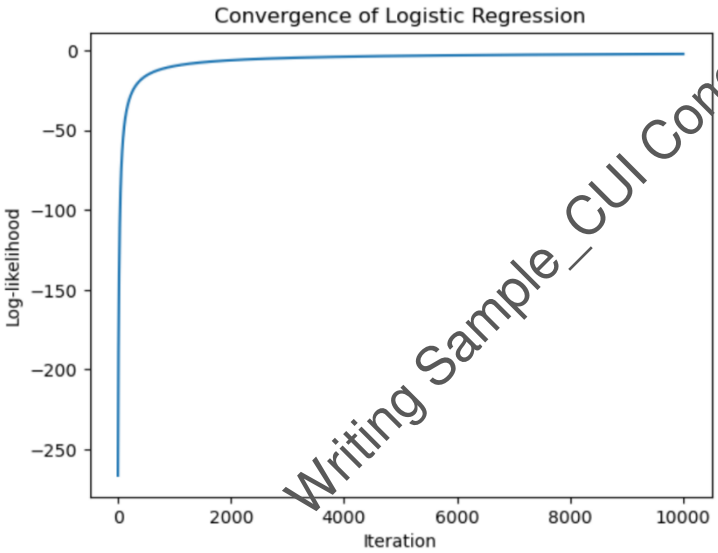
Although different learning rates and iteration counts produced **different** parameter vectors, the classification performance on the test dataset remained equivalent as long as the accuracy was the **same**. In other words, multiple solutions can represent the same decision boundary in logistic regression. However, using **fewer iterations is more efficient,** since the model can reach perfect accuracy with less computational effort.

From the numerical results alone, it is not possible to determine which configuration is superior. The choice should be guided by the convergence behavior observed in the **log-likelihood curve.** The curve, shown below, illustrates the degree of convergence and stability, and thus provides the basis for selecting the most appropriate learning rate**.(e)**

(1) Learning rate = 0.001, Iteration = 20000

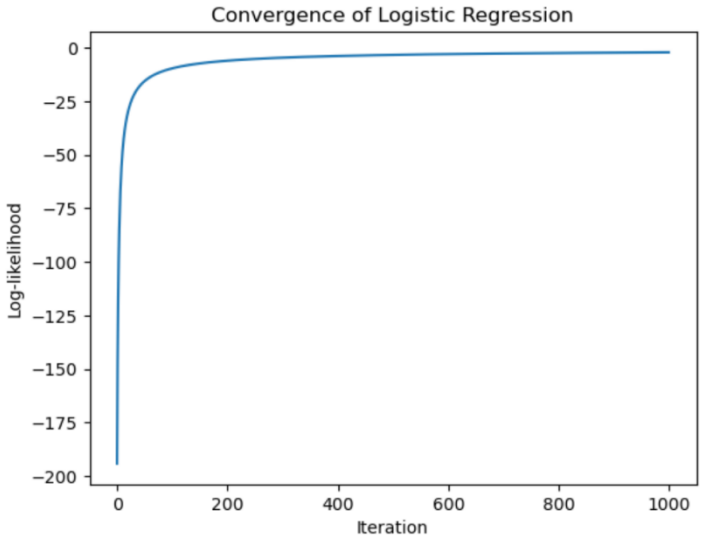


(2) Learning rate = 0.01, Iteration = 10000

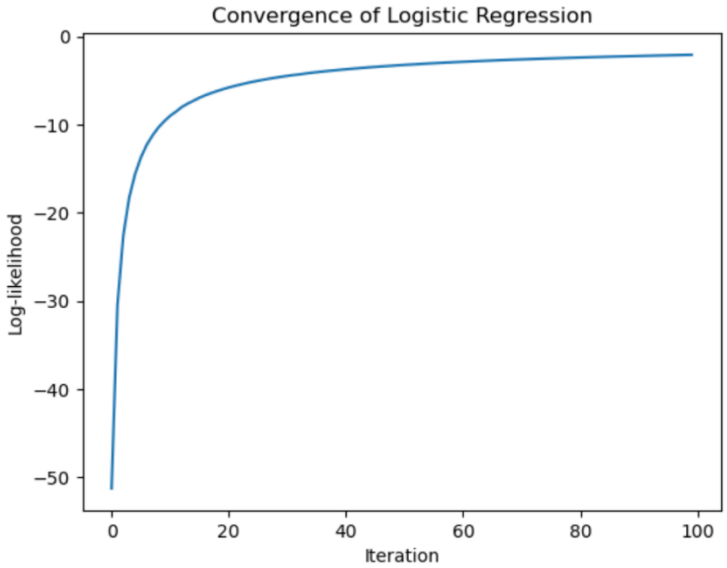


Writing Sample_CUI Congwen

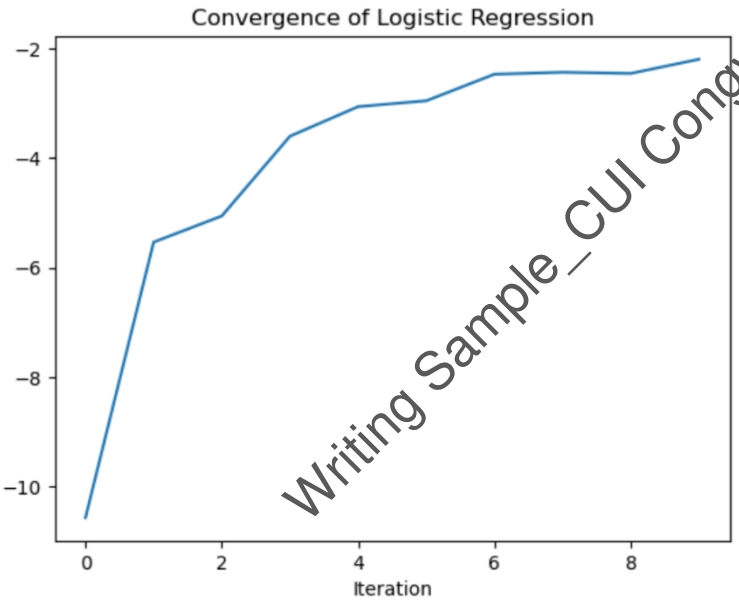
(3) Learning rate = 0.01, Iteration =1000



(4) Learning rate = 0.1, Iteration = 100



(5) Learning rate = 1, Iteration = 10



Writing Sample_CUI Congwen

According to the figures, we found that there is no significant difference between smaller and moderate learning rate. But larger learning rates introduced oscillations in the log-likelihood curve, reflecting instability in the update process. Although the final accuracy was still perfect, such unstable convergence is not reliable in more complex datasets.

Configuration (4), with a learning rate of α = **0.1 and 100 iterations**, **was selected as the final parameter vector** ***w = [1.2274, -6.5175, 9.4390, -6.7356, 8.4234, -4.8227, 0.1201]*****.** This choice is justified because it reached perfect accuracy without oscillations and required fewer iterations compared to smaller learning rates. In other words, it provided the most efficient balance between stability and computational cost.

Dataset Simplicity and Its Implications

The fact that very low iteration counts still produced perfect accuracy raised the suspicion that the dataset might be so simple that a single hyperplane could already separate all points.

To test this hypothesis, I deliberately reduced the number of iterations to 1 and experimented with extremely large learning rates to see whether the updates would overshoot the solution.

Learning rate (α)	Iterations	Accuracy	Misclassified
1000	1	0.97	3
100		1.0	0
10		0.95	5
5		1.0	0

The detailed testing result is shown below:

(1) Learning rate = 1000

```
Number of misclassified: 3 / 100
w: [  494.01985219 -2551.23385856  4447.60374226 -2252.98953754
  3295.2500977   -754.78411825   499.33984819]
Test accuracy: 0.97
```

(2) Learning rate = 100

```
Number of misclassified: 0 / 100
w: [  34.62430965 -142.89550381  223.14826671 -161.3713206   132.48947439
  -27.10023406   -3.67231993]
Test accuracy: 1.0
```

(3) Learning rate = 10

```
Number of misclassified: 5 / 100
w: [  8.02199619 -19.33879124  24.80991234 -14.54133456  19.5078302
  -1.64931299  -0.8065126 ]
Test accuracy: 0.95
```

(4) Learning rate = 5

```
Number of misclassified: 0 / 100
w: [ 1.35306819 -7.53751063 11.93564431 -9.02796898  8.46875734 -3.89468298
  0.52145792]
Test accuracy: 1.0
```

Writing Sample _ CUI Congwen

The results confirmed that although the parameter values varied dramatically, the classification accuracy remained high, and in some cases perfect. This indicates that the direction of the parameter vector was sufficient to define a separating hyperplane, and the dataset was indeed linearly separable. **In other words, the simplicity of the dataset allowed logistic regression to converge almost immediately**, regardless of the scale of the update step.

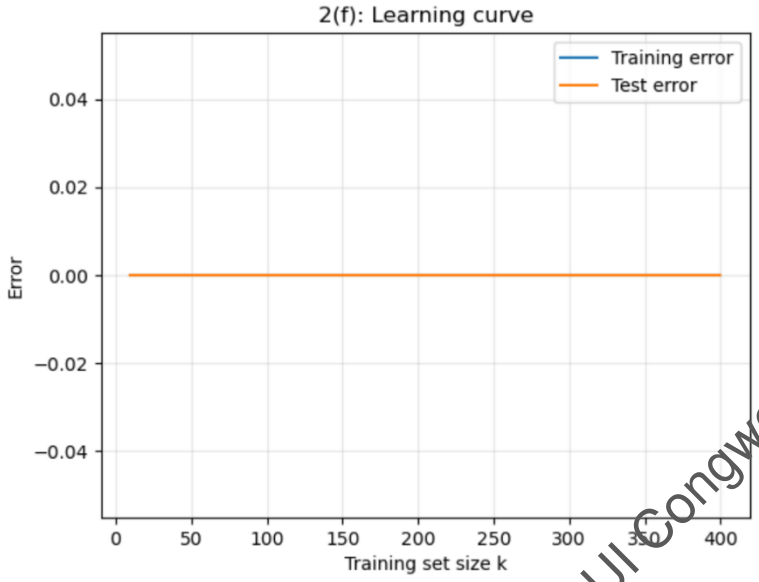
To prove the result, I double check the dataset and I found the training set contains 400 samples with 6 features each, while the testing set contains 100 samples with the same structure. The labels are almost perfectly separable: samples with certain feature combinations consistently belong to one class, while the opposite combinations belong to the other.

This structural analysis demonstrates that the dataset is linearly separable and relatively small, which explains why logistic regression achieved 100% accuracy under all tested configurations.

(f) Effect of Training Set Size

To investigate the impact of **training set size** on model performance, we randomly selected subsets of size $k \in \{10, 20, \ldots, 400\}$ from the original training data. For each subset, the logistic regression classifier was retrained using stochastic gradient ascent, and both the training error (on the selected subset) and the test error (on the full testing dataset) were recorded.

The resulting learning curves are shown in Figure below.



**The two lines coincide.*

Writing Sample — CUI Congwen

The learning curve shows that both training and test errors remain at zero across all training set sizes. This outcome reflects the **simplicity and linear separability** of the dataset: even with very small subsets, the logistic regression classifier quickly converges to a separating hyperplane that achieves perfect accuracy. As a result, the training error curve overlaps with the test error curve, both lying flat at zero.

This confirms that the dataset is linearly separable and that logistic regression is capable of achieving reliable classification with minimal computational effort.